

안심식품 검색기

빌드 & 배포 가이드 · v1.0 · 2026-05-31

이 문서는 **안심식품 검색기** 풀스택 서비스를 로컬 개발 환경에서 프로덕션으로 옮길 때 필요한 단계를 정리합니다. React/Vite 프론트엔드와 Node.js/Express 백엔드, 그리고 식품의약품안전처 OpenAPI 연동 + 매일 04:00 동기화까지 한 번에 다룹니다.

1. 시스템 구조 한눈에 보기

프론트엔드는 Vite로 빌드된 정적 자산이고, 백엔드는 Express + node-cron 프로세스입니다. 데이터베이스가 없으며 식약처에서 받은 회수 이력은 메모리와 backend/data/recalls.json에 영속화됩니다.

구성요소	역할	프로세스
frontend/dist	정적 자산 (HTML/JS/CSS)	Nginx · Vercel · Express static 등
backend (Express)	검색 API + 동기화 cron	PM2 · Docker · systemd
backend/data/recalls.json	회수 이력 캐시 (~355건)	백엔드가 읽기/쓰기
backend/logs/sync.log	동기화 시도/결과 누적	운영자만 봄 (UTF-8)
식약처 OpenAPI	원본 데이터 출처	매일 04:00 증분 호출

2. 필수 준비물

- Node.js 18 이상 (LTS 권장)
- npm 9 이상 (Node와 함께 설치됨)
- 식약처 OpenAPI 인증키 — data.go.kr 또는 foodsafetykorea.go.kr에서 발급. 개발계정은 자동 승인이라 즉시 가능.
- (선택) Discord 웹훅 URL — 동기화 실패 시 즉시 알림
- (프로덕션) 도메인 + HTTPS 인증서 — Let's Encrypt 권장

3. 로컬 빌드 — 정적 자산 만들기

프론트엔드를 정적 자산으로 빌드합니다. 결과물은 frontend/dist/에 떨어지며 이것 그대로 정적 호스팅에 올리면 됩니다.

```
cd frontend
npm install      # only first time
npm run build
# -> outputs to frontend/dist/index.html, assets/*
```

참고 Vite의 dev 서버(npm run dev)는 /api/* 요청을 backend(3001)로 프록시하지만, 프로덕션 빌드된 정적 자산은 프록시 설정을 들고 가지 않습니다. → 다음 장의 호스팅 방식 중 하나로 /api/* 라우팅을 맞춰주세요.

4. 정적 자산 호스팅 — 3가지 선택지

서비스 규모와 예산에 따라 적절한 방식을 선택합니다. 복잡도가 가장 낮은 순서로 정리했습니다.

옵션 A. 백엔드가 정적 자산까지 서빙 (1대 서버)

Express에 `express.static` 한 줄만 추가하면 프론트엔드를 백엔드 같은 포트에서 서빙합니다. 가장 단순하고 무료. 소규모 트래픽이거나 개인 프로젝트라면 충분합니다.

```
// Append to backend/src/server.js
const path = require('path');
app.use(express.static(path.resolve(__dirname, '../..../frontend/dist')));
app.get('*', (req, res) => {
  res.sendFile(path.resolve(__dirname, '../..../frontend/dist/index.html'));
});
```

- 장점: 추가 인프라 없음, CORS 신경 안 써도 됨, `/api/*`도 같은 origin
- 단점: Node 프로세스가 정적 파일까지 처리 (트래픽 많아지면 비효율)

옵션 B. Vercel · Netlify (프론트만 별도 호스팅)

프론트엔드는 Vercel/Netlify의 무료 정적 호스팅에 두고, 백엔드는 별도 서버(VPS 등)에 둡니다. CDN 캐싱 자동, 글로벌 전송 빠름. API URL을 환경변수로 주입해야 합니다.

```
# Build with API origin injected as env var
VITE_API_BASE=https://api.example.com npm run build
```

그리고 `src/constants/config.js`에서 `import.meta.env.VITE_API_BASE`를 읽도록 수정합니다. 백엔드는 `cors({ origin: 'https://your-site.vercel.app' })`로 origin을 좁혀주세요.

옵션 C. Nginx + PM2 (자체 VPS / 클라우드)

표준 프로덕션 구성. Nginx가 정적 자산을 직접 서빙하고 `/api/*` 만 백엔드로 reverse proxy. HTTPS 종단도 Nginx가 담당.

```
# /etc/nginx/sites-available/food-safety.conf
server {
  listen 443 ssl http2;
  server_name food-safety.example.com;

  ssl_certificate      /etc/letsencrypt/live/.../fullchain.pem;
  ssl_certificate_key  /etc/letsencrypt/live/.../privkey.pem;

  root /var/www/food-safety/dist;
  index index.html;

  # SPA fallback -- serve index.html on refresh
  location / {
    try_files $uri $uri/ /index.html;
  }

  # Backend API
```

```
location /api/ {
    proxy_pass http://127.0.0.1:3001;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}
}
```

- HTTPS는 Let's Encrypt(certbot)로 무료 발급, 자동 갱신
- 프론트 빌드는 cron 또는 CI에서 빌드 후 dist/만 rsync

5. 백엔드를 영구 실행하기

PM2로 데몬화

터미널을 닫아도 계속 돌고, 죽으면 자동 재시작합니다. 메모리 누수 시 일정 시점에 재시작도 설정 가능.

```
npm install -g pm2
cd backend
pm2 start src/server.js --name food-safety-api
pm2 save
pm2 startup          # auto-start on system boot
```

- 로그 확인: `pm2 logs food-safety-api`
- 상태 확인: `pm2 status`
- 재시작: `pm2 restart food-safety-api`

Docker로 컨테이너화 (대안)

PM2 대신 컨테이너 표준을 따르고 싶다면. 한 가지 주의: backend/data/와 backend/logs/는 볼륨으로 빼야 컨테이너 재시작 시 데이터가 유지됩니다.

```
# backend/Dockerfile
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY src ./src
EXPOSE 3001
CMD ["node", "src/server.js"]

# Run
docker build -t food-safety-api ./backend
docker run -d --name fsa \
  -p 3001:3001 \
  --env-file ./env \
  -v $(pwd)/backend/data:/app/data \
  -v $(pwd)/backend/logs:/app/logs \
  food-safety-api
```

6. 프로덕션 환경 설정 (.env)

C:/food-safety/.env.example을 .env로 복사한 뒤 값을 채웁니다. 키는 절대 git에 커밋하지 마세요 — .gitignore에 .env가 이미 포함되어 있습니다.

```
FOODSAFETY_API_KEY=your_actual_key_here
DISCORD_WEBHOOK_URL=https://discord.com/api/webhooks/...
PORT=3001
SYNC_CRON=0 4 * * *
```

- 개발계정 API는 일일 10,000건 제한 — 본 서비스는 1일 ~10건만 호출하므로 충분
- Discord 웹훅을 비워두면 알림이 건너뛰어지고 로그 파일에만 기록
- SYNC_CRON 형식: 분 시 일 월 요일 (예: 0 4 * * * = 매일 04:00)

7. 운영 점검 체크리스트

배포 직후

- **최초 동기화 성공 확인** — 로그에 "동기화 성공 | 유형: 전체 | 건수: N건" 1줄
- GET /api/status 호출 → count, lastUpdated 응답
- 브라우저로 진입 → 홈 화면 + "마지막 업데이트" 표시 확인
- 실제 검색어로 결과 1건 이상 노출 확인

매일

- 04:00 cron 동기화 → 로그 마지막 라인이 당일 04:00인지
- Discord 알림 채널 — 전일 실패 메시지 없는지

주기적 (월 1회)

- backend/logs/sync.log 크기 — 너무 크면 로테이션 (winston + daily-rotate-file)
- 의존성 보안 업데이트 — npm audit fix
- 프론트 정적 자산 재빌드 + 배포 (의존성 메이저 업데이트가 있었을 때)

8. 문제 해결 (Troubleshooting)

증상	원인 / 해결
서버 시작 시 "동기화 실패"	API 키 오타(INFO-100) · 인터넷 차단 · 식약처 서버 장애. logs/sync.log에서 코드 확인.
검색 결과 0건만 나옴 (정상 데이터가 있는데)	recalls.json이 비어있거나 손상. 서버 재시작 시 자동 전체 동기화가 트리거됩니다.
프론트에서 /api/* 가 404	프록시 미설정 (옵션 B/C). Nginx location /api/ 또는 환경변수 VITE_API_BASE 확인.
다크 모드 깜빡임	정적 자산은 첫 페인트가 라이트 토큰. index.html에 inline script로 data-theme 즉시 적용 가능.
새로고침 시 404	정적 호스팅에서 SPA fallback 미설정. Nginx의 try_files \$uri /index.html; 등 추가.
메모리 증가 (PM2)	현재 데이터 규모(1MB 미만)는 무관. node 캐시일 수 있으니 24h 단위 자동 재시작 설정.

9. 보안 체크 — 배포 전 한 번 더

- .env가 .gitignore에 들어가 있는지 — git status로 확인
- API 키가 코드/로그에 평문 노출되지 않는지 — 로그는 ***KEY***로 마스킹됨
- 백엔드 CORS — 옵션 B처럼 다른 origin일 때만 origin 좁히기 (옵션 A/C는 같은 origin이라 무관)
- HTTPS 강제 — Nginx에서 80→443 리다이렉트

- 백엔드 포트(3001)는 외부 노출 X — 방화벽에서 차단, Nginx만 접근
- PM2 / Docker가 root 권한이 아닌 일반 유저로 실행되는지

부록. 주요 파일 위치

경로	설명
.env	환경 변수 (절대 커밋 X)
.env.example	환경 변수 템플릿 (커밋 OK)
backend/src/server.js	진입점, cron 등록
backend/src/config.js	.env 값 모음
backend/src/services/sync.js	식약처 동기화
backend/src/services/notify.js	로그 + Discord 알림
backend/data/recalls.json	회수 이력 캐시 (자동 생성)
backend/logs/sync.log	동기화 시도 누적 로그
frontend/dist/	프로덕션 빌드 결과물
frontend/src/constants/	한국어 문자열, 등급, API 설정

이 가이드는 v1.0 풀스택 구현 (백엔드 Express + 프론트 React/Vite) 기준입니다. 더 자세한 코드 흐름은 프로젝트 메모리([claude/projects/.../memory/](#))와 각 모듈 상단 주석을 참고하세요.